API	Description
	you can continue to make use of XInput. XInput has been replaced by Windows.Gaming.Input for UWP, and if you're writing new input code, you should use Windows.Gaming.Input instead of XInput. **For more information** See the XInput documentation.
Windows.Gaming.Input	The Windows.Gaming.Input API replaces XInput and provides the same functionality with the following advantages over Xinput: ◦ Lower resource usage ◦ Lower API call latency for retrieving input ◦ The ability to work with more than 4 gamepads at once ◦ The ability to access additional gamepad features, such as the trigger vibration motors ◦ The ability to be notified when controllers connect/disconnect via event instead of polling ◦ The ability to attribute input to a specific user (Windows.System.User) **When to use** If your game needs to support gamepad input and is not using existing XInput code or you need one of the benefits listed above, you should make use of Windows.Gaming.Input. **For more information** See the Windows.Gaming.Input documentation.
Windows.UI.Core.CoreWindow	The Windows.UI.Core.CoreWindow class provides events for tracking pointer presses and movement, and key down and key up events. **When to use** Use Windows.UI.Core.CoreWindows events when you need to track the mouse or key presses in your game.

API	Description
	For more information See Move-look controls for games for more information about using the mouse or keyboard in your game.

- Math - APIs concerning simplifying commonly used mathematical operations.

Expand table

API	Description
DirectXMath	The DirectXMath API provides SIMD-friendly C++ types and functions for common linear algebra and graphics math operations common to games. **When to use** Use of DirectXMath is optional and simplifies common mathematical operations. **For more information** See the DirectXMath documentation.

- Networking - APIs concerning communicating with other computers and devices over either the Internet or private networks.

Expand table

API	Description
Windows.Networking.Sockets	The Windows.Networking.Sockets namespace provides TCP and UDP sockets that allow reliable or unreliable network communication. **When to use** Use Windows.Networking.Sockets if your game needs to communicate with other computers or devices over the network.

API	Description
	For more information See Work with networking in your game.
Windows.Web.HTTP	The Windows.Web.HTTP namespace provides a reliable connection to HTTP servers that can be used to access a web site. **When to use** Use Windows.Web.HTTP when your game needs to access a web site to retrieve or store information. **For more information** See Work with networking in your game.

- Support Utilities - Libraries that build on the Windows 10 APIs.

Expand table

Library	Description
DirectX Tool Kit	The DirectX Tool Kit (DirectXTK) is a collection of helper classes for writing DirectX 11.x code in C++. **When to use** Use the DirectX Tool Kit if you're a C++ developer looking for a modern replacement to the legacy D3DX utility code or you're an XNA Game Studio developer transitioning to native C++. **For more information** See the DirectX Tool Kit project page, https://github.com/Microsoft/DirectXTK .
Win2D	Win2D is an easy-to-use Windows Runtime API for immediate mode 2D graphics rendering. **When to use**

Library	Description
	Use Win2D if you're a C++ developer and want an easier to use WinRT wrapper for Direct2D and DirectWrite, or you're a C# developer wanting to use Direct2D and DirectWrite. **For more information** See the Win2D project page, https://github.com/Microsoft/Win2D.

Xbox Live Services

The Xbox Developer Programs allow any developer to integrate Xbox Live into their UWP game and publish to Xbox One and Windows 10. Integrate Xbox Live social experiences such as sign-in, presence, leaderboards, and more into your title, with minimal development time. Xbox Live social features are designed to organically grow your audience, spreading awareness to over 55 million active gamers.

If you want access to even more Xbox Live capabilities, dedicated marketing and development support, and the chance to be featured in the main Xbox One store, apply to the ID@Xbox program. To see which features are available to the Xbox Live Creators Program and ID@Xbox program, see the Feature table.

For more info, go to Adding Xbox Live to your game.

Alternatives to writing games with DirectX and UWP

UWP games without DirectX

Simpler games with minimal performance requirements, such as card games or board games, can be written without DirectX and don't necessarily need to be written in C++. These sort of games can make use of any of the languages supported by UWP such as C#, Visual Basic, C++, and HTML/JavaScript. If performance and intensive graphics are not a requirement for your game, checkout JavaScript and HTML5 touch game sample as an example.

Game engines

As an alternative to writing your own game engine using the Windows game development APIs, many high quality game engines that build on the Windows game development APIs are available for developing games on Windows platforms. When considering a game engine or library, you have multiple options:

- Full game engine - A full game engine encapsulates most or all of the Windows 10 APIs you would use when writing a game engine from scratch, such as graphics, audio, input, and networking. Full game engines may also provide game logic functionality such as artificial intelligence and pathfinding.
- Graphics engine - Graphics engines encapsulate the Windows 10 graphics APIs, manage graphics resources, and support a variety of model and world formats.
- Audio engine - Audio engines encapsulate the Windows 10 audio APIs, manage audio resources, and provide advanced audio processing and effects.
- Network engine - Network engines encapsulate Windows 10 networking APIs for adding peer-to-peer or server-based multiplayer support to your game, and may include advanced networking functionality to support large numbers of players.
- Artificial intelligence and pathfinding engine - AI and pathfinding engines provide a framework for controlling the behavior of agents in your game.
- Special purpose engines - A variety of additional engines exist for handling almost any game development related task you might run into, such as creating inventory systems and dialog trees.

Submitting a game to the Microsoft Store

Once you're ready to publish your game, you'll need to create a developer account and submit your game to the Microsoft Store.

For information about submitting your game to the Microsoft Store, see Submitting and publishing your game.

Feedback

Making games accessible

Article • 03/16/2023

Accessibility can empower every person and every organization on the planet to achieve more, and this applies to making games more accessible too. This article is written for game developers, game designers, and producers. It provides an overview of game accessibility guidelines derived from various organizations (listed in the reference section below), and introduces the inclusive game design principle for creating more accessible games.

Gaming for Everyone

At Microsoft, we believe that gaming should be fun for everyone. We "felt compelled to do more to make gaming an inclusive environment that embraced everyone. We fundamentally believe that what we build for our fans and the way we show up – inside and outside the walls of Microsoft – is a reflection of who we are. We designed the program to reflect the core values we have as an organization and believe that the program could result in positive change – not only in our workplace but in the products we build for the gamers we serve." (Blog post from Phil Spencer)

We want to create a fun, diverse, and inclusive environment where everyone can play in. "To truly have a lasting impact requires a culture shift, one that won't happen overnight. However, our team is committed to get better each day, to teach one another to pause in our decision making process and think about the amazing diversity of needs, abilities and interests amongst gamers around the world." (Blog post from Phil Spencer)

We hope you'll join us on this journey to make Gaming for Everyone happen.

Why make games accessible?

Increased gamer base

At its most basic level, the business justification for accessibility is straightforward:

Number of users who can play your game x Awesomeness of game = Game sales

If you made an amazing game that is so complicated or convoluted that only a handful of people can play it, you limit your sales. Similarly, if you made a game that is unplayable by those with physical, sensory, or cognitive impairments, you are missing out on potential sales. Considering that, for example, 19% of people in the United States

have some form of disability, estimated 14% of adults in the US have difficulty reading, and estimated 10% of males have some form of color vision deficiency, this can potentially have a large impact on your title's revenue.

For more business justifications, see Making Video Games Accessible.

Better games

Creating a more accessible game can create a better game in the end.

An example is subtitles in games. In the past, games rarely supported subtitles or closed captioning for game dialogues. Today, it's expected that games include subtitles and closed captioning. This change was not driven by gamers with disabilities. Instead, it was driven by localization, but became popular with a wide range of gamers who simply preferred to play with subtitles because it made the gaming experience better. Gamers turn subtitles and closed captioning on when they are playing with too much background noise, are having difficulty hearing voices with various sound effects or ambient sounds playing at the same time, or when they simply need to keep the volume low to avoid disturbing others. Subtitles and closed captions not only helped gamers to have a better gaming experience, but it also allows people with hearing disabilities to game as well.

Controller remapping is another feature that is slowly becoming a standard for the game industry for similar reasons. It is most commonly offered as a benefit for all players. Some gamers enjoy customizing their gaming experiences and some simply prefer something different from what the designers had in mind. What most people don't realize is that the ability to remap buttons on an input device is actually also an accessibility feature that was intended to make a game playable for people with various types of motor disabilities, who are physically unable to or find it difficult to operate certain areas of the controller.

Ultimately, the thought process used to make your game more accessible will often result in a better game because you have designed a more user-friendly, customizable experience for your players to enjoy.

Social space and quality of life

Video games are one of the highest grossing forms of entertainment and gaming can provide hours of joy. For some, gaming is not only a form of entertainment but it is an escape from a hospital bed, chronic pain, or debilitating social anxiety. Gamers are transported into a world where they become the main characters in the video game. Through gaming, they can create and participate in a social space for themselves that

provides distraction from the day-to-day struggles brought on by their disabilities, and that provides an opportunity to communicate with people they might otherwise be unable to interact with.

Gaming is also a culture. Being able to take part in the same thing that all of your friends are talking about is something that can be hugely valuable to someone's quality of life.

Is the game you are making today accessible?

If you are thinking about making your game accessible for the first time, here are some questions to ask yourself:

- Can you complete the game using a single hand?
- Can an average person be able to pick the game up and play?
- Can you effectively play the game on a small monitor or TV sitting at a distance?
- Do you support more than one type of input device that can be used to play through the entire game?
- Can you play the game with sound muted?
- Can you play the game with your monitor set to black and white?
- When you load your last saved game after a month, can you easily figure out where you are in the game and know what you need to do in order to progress?

If your answers are mostly no, or you do not know the answers, it is time to step up and put accessibility into your game.

Defining disability

Disability is defined as "a mismatch between the needs of the individual and the service, product or environment offered." (Inclusive video, Microsoft.com.) This means that anyone can experience a disability, and that it can be a short-term or situational condition. Envision what challenges gamers with these conditions might have when playing your game, and think about how your game can be better designed for them. Here are some disabilities to consider:

Vision

- Medical, long-term conditions like glaucoma, cataracts, color blindness, near-sightedness, and diabetic retinopathy
- Short-term, situational conditions like a small monitor or screen size, a low resolution screen, or screen glare due to bright light sources like the sun on a monitor or mobile screen

Hearing

- Medical, long-term conditions like complete deafness or partial hearing loss due to diseases or genetics
- Short-term, situational conditions like excessive background noise, low quality audio quality, or restricted volume to avoid disturbing others

Motor

- Medical, long-term conditions like Parkinson's disease, amyotrophic lateral sclerosis (ALS), arthritis, and muscular dystrophy
- Short-term, situational conditions like an injured hand, holding a beverage, or carrying a child in one arm

Cognitive

- Medical, long-term conditions like dyslexia, epilepsy, attention deficit hyperactivity disorder (ADHD), dementia, and amnesia
- Short-term, situational conditions like alcohol consumption, lack of sleep, or temporary distractions like siren from an emergency vehicle driving by the house

Speech

- Medical, long-term conditions like vocal cord damage, dysarthria, and apraxia
- Short-term, situational conditions like dental work, or eating and drinking

How to make games more accessible?

Design shift: Inclusive game design approach

Inclusive design focuses on creating products and services more accessible to a broader spectrum of consumers, including people with disabilities.

To be successful, today's game designers need to think beyond creating games that they enjoy. Game designers need to be aware of how their design decisions impact the overall accessibility of the game; the playability of the game for their entire target audience, including those with disabilities.

As such, traditional game design paradigms must shift to embrace the inclusive game design concept. Inclusive game design means going beyond the basic game design of

creating fun for the target audience, to creating additional or modified personas to include a wider spectrum of players. You need to be acutely aware about designing barriers in your game and make sure that they do not add unnecessary barriers that take the fun away from the intended experience.

By identifying gaps, you can optimise, iterate on the original design concept and make it better, allowing more people to experience your vision. When you take the time to be more inclusive in your game design process, your final game becomes more accessible. No game can ever work for everyone, the definition of game requires that there is some degree of challenge, but through considering accessibility you can ensure that no one is unnecessarily excluded.

Empower gamers: Give gamers options

Nearly every accessibility solution comes down to one of two principles. The first is giving your gamers the options to customize their gaming experience. If you already have a huge fan base, you may have a significant portion of your audience who do not want the experience to change in any way. That's okay. Give your gamers the ability to turn these features on and off, and make features configurable individually. You need to allow people to experience the game in the way that best suits their own needs and preferences.

Reinforce: Communicate information in more than one way

The second principle is where the concept of universal design comes in, a single approach that not only brings in more players but also improves the experience for all. For example an image as well as text, a symbol as well as colour. A map that is based on a range of different coloured markers is not only impossible for color blind gamers to use, it is also frustrating for everyone else who must remember what everything equates to. Adding symbols makes it a better experience for everyone.

Innovate: Be creative

There are many creative ways to improve the accessibility of your game. Put on your creative hat and learn from other accessible games out there. If you already have an existing game, learn to identify current game features that could be improved while keeping the core game mechanics and experience as designed. As mentioned above, accessibility in games is all about providing gamers with options to customize their

gaming experience. It could be through reinforcement or communicating information in more than one way.

Considering accessibility allows you to approach design from a new angle and possibly ideas you would have not thought of otherwise. This approach to design resulted not only in interesting concepts but have created products that have wide spread adoption or mass market commercial success. Examples include predictive text, voice recognition, curb cuts, the loudspeaker, the typewriter, and Optical Character Recognition (OCR). Ideas for these products came from those who started thinking about solutions for accessibility.

Adopt: Quality means accessible features

Accessibility is a measure of quality. It has to be a feature requirement and not a good-to-have work item. For example, "Adapt minimap for colourblindness" is not considered a low priority work item that you get to if you have extra time. If this work item is not done, it simply means that entire minimap feature is incomplete and cannot be shipped.

Evangelize: Make accessibility a priority in your game studio

Game development is always running on a tight timeline, so prioritizing accessibility will help make it an easier process. One way is to design from the start with accessibility in mind. The earlier you consider accessibility, the easier and cheaper it becomes.

Share your knowledge about accessibility with your team, share the business justifications, and dispel the common misconceptions – that it doesn't benefit many people, it dilutes your mechanic, and it's difficult and expensive to implement.

Review: Constantly evaluate your game

During development, you can introduce a review process to make sure that at every step of the way you are thinking about accessibility. Make a checklist like the one below to help your team constantly evaluate whether what you are creating is accessible or not.

Checklist	**Accessibility features**
In-game cinematics	Has subtitles and captions, photosensitivity tested
Overall artwork (2D and 3D graphics)	Color blind friendly colors and options, not dependent entirely on color for identification but use shapes and patterns as well

Checklist	Accessibility features
Start screen, settings menu and other menus	Ability to read options aloud, ability to remember settings, alternate command control input method, adjustable UI font size
Gameplay	Wide adjustable difficulty levels, subtitles and captions, good visual and audio feedback for gamer
HUD display	Adjustable screen position, adjustable font size, color blind friendly option
Control input	Mappable controls to input device, custom controller support, simplified input for game allowed

Playtest and iterate: Get gamers' feedback

When organizing playtesting sessions, invite play testers with disabilities that your game is designed for and get them to play your game. Remember to include accessibility questions in the Beta test questionaires. Local disability groups are a great source of participants. Observe how they play and get feedback from them. Figure out what changes need to be made to make the game better.

Use social media and your game's forum to listen for input about which accessibility features matter most and how they should be implemented.

Shout it out: Let the world know your game is accessible

Consumers will want to know if your game can be played by gamers with disabilities. State the game's accessibility clearly on the game website, press releases, and packaging to ensure that consumers know what to expect when they buy your game. Remember to make your website and all sales channels to the game accessible as well. Most importantly, reach out to the accessibility gaming community and tell them about your game.

Game accessibility features

This section outlines some features that can make your game more accessible. These features are derived from guidelines taken from the Game accessibility guidelines website. That resource represent the findings of a collaborative group of studios, specialists, and academics.

Color blind friendly graphics and user interface

The retina of the eye has two types of light-sensitive cells: the cones for seeing where there is light, and the rods for seeing in low light conditions.

There are three types of cones (red, green, and blue) to enable us to view colors correctly. Color blindness occurs when one or more of these three types light cones is not functioning as expected. The degree of color blindness can range from almost normal color perception with reduced sensitivity towards red, green, or blue light, to a complete inability to perceive any color at all.

Since it's less common to have reduced sensitivity to blue light, when designing for the color blind, the selection of colors are geared towards people who are red or green color blind:

- Use color combinations that can be differentiated by people with red/green color blindness:
  - Colors that appear similar: All shades of red and green including brown and orange
  - Colors that stand out: Blue and yellow
- Do not rely solely on color to communicate or distinguish game objects. Use shapes and patterns as well.
- If you have to rely on colors alone, combine presets with a free selection of colors, so that it can be fully customizable by the players who need them and not creating extra work for players who do not need them.
- Use a color blind simulator to test your designs so that you can view your designs through color blind eyes. This can help you avoid common contrast issues. Color Oracle is a free color blind simulator that can simulate the three most common types of color vision deficiency – deuteranopia, protanopia, and tritanopia.

Closed captioning and subtitles

When designing the closed captions and subtitles for your game, the objective is provide readable captions as an option so that your game can also be enjoyed without audio. It should be possible to have game components like game dialogues, game audio, and sound effects displayed as text on screen.

Here are some basic guidelines to consider when designing closed captions and subtitles:

- Select simple readable font.

- Select sufficiently large font size, or consider having adjustable font size option for more flexibility. (Ideal font size depends on screen size, viewing distance from screen, and so on.)
- Create high contrast between background and font color. Use strong outline and shadows for the text. Use a dark background overlay for the captions and remember to provide options for it to be turned on or off. (For more information, see Information on contrast ratio.)
- Display short sentences on screen, maximum 38 characters per line and maximum 2-3 lines at any one time. (Remember not to give the game away by displaying the text before event occurs.)
- Differentiate what is making the sound or who is talking. (Example: "Daniel: Hi!")
- Provide the option to turn closed captions and subtitles on and off. (Additional feature: Ability to select how much sound information is displayed based on importance.)

Game chat transcription

If your title allows gamers to communicate using voice and send text messages to one another, Text-to-Speech and Speech-to-Text functionalities should be available as an option.

People who do not have microphones attached to their gaming device can still have a voice conversation with someone who is speaking. They are able to type text into the chat window and have those messages converted into voice. It also allows someone who can't hear very well to read the transcribed text messages from the person they're having a voice chat with.

For developers in the ID@Xbox and managed partners program, Text-to-Speech and Speech-to-Text features are available as part of the Game Chat 2 accessibility features in the Xbox Live service. For more information, see Game Chat 2 Overview.

Sound feedback

Sound provides feedback to the player, in addition to visual feedback. Good game audio design can improve accessibility for players with visual impairment. Here are some guidelines to consider:

- Use 3D audio cues to provide additional spatial information.
- Separate music, speech and sound effects volume controls.
- Design speech that provides meaningful information for gamers. (Example: "Enemies are approaching" vs. "Enemies are entering from the back door.")

- Ensure speech is spoken at a reasonable rate, and provide rate control for better accessibility.

Fully mappable controls

There are companies and organizations, such as SpecialEffect, that design custom game controllers that can be used with various gaming systems such as Windows and Xbox devices. This customization allows people with different forms of disabilities to play games that they might not be able to play otherwise. For more information on people who are now able to play games independently because of customized controllers, see Whom we've helped on the SpecialEffect website.

As a game developer, you can make your game more accessible by allowing fully mappable controls so that gamers have the option to plug in their custom controllers and remap the keys according to their needs.

Having fully mappable controls also benefits people who use standard controllers. Your gamers can design a layout that suits their unique individual needs.

Both standard Xbox One and Xbox Elite controllers offer customization of the controllers for precision gaming. To fully utliize their remapping capabilities, **it is recommended that developers include remapping directly in the game**. For more information, see Xbox One and Xbox Elite.

Wider selection of difficulty levels

Video games provide entertainment. The challenge for game developers is to tune the difficulty level such that the gamer experiences the right amount challenge. Firstly, not all gamers have the same skill level and capability, so designing a wider selection of difficulty options increases the chance of providing gamers with the right amount of challenge. At the same time, this wider selection also makes your video game more accessible because it could potentially allow more people with disabilities to play your game. Remember, gamers want to overcome challenges in a game and be rewarded for it. They do not want a game that they cannot win.

Tweaking the difficulty level of your game is a delicate process. If it is too easy, gamers might get bored. If it is too difficult, gamers may give up and not play any further from that point on. The balancing process is both art and science. There are many ways to make a game level that has the right amount of challenge. Some games offer simplified inputs, like a single button press game option for their game, a rewind and replay option to make gameplay more forgiving, or less and weaker enemies to make it easier to proceed forward after several tries.

Photosensitivity epilepsy testing

Photosensitive epilepsy (PSE) is a condition where seizures are triggered by visual stimuli, including exposures to flashing lights or certain moving visual forms and patterns. This occurs in about three percent of people and is more common in children and adolescents. In terms of numbers, we are looking at approximately 1 in 4000 people aged 5-24.

There are many factors that can cause a photosensitive reaction when playing video games, including the duration of gameplay, the frequency of the flash, the intensity of the light, the contrast of the background and the light, the distance between the screen and the gamer, and the wavelength of the light.

Many people discover that they have epilepsy through having a seizure. Gamers can and do have their first seizures through videogames, and this can result in physical injury. As a developer, here are some tips for designing a game to reduce the risk of seizures caused by photosensitive epilepsy.

Avoid the following:

- Having flashing lights with a frequency of 5 to 30 flashes per second (Hertz) because flashing lights in that range are most likely to trigger seizures.
- Any sequence of flashing images that lasts for more than 5 seconds
- More than three flashes in a single second, covering 25%+ of the screen
- Moving repeated patterns or uniform text, covering 25%+ of the screen
- Static repeated patterns or uniform text, covering 40%+ of the screen
- An instantaneous high change in brightness/contrast (including fast cuts), or to/from the colour red
- More than five evenly spaced high contrast repeated stripes – rows or columns such as grids and checkerboards, that may be composed of smaller regular elements such as polkadots
- More than five lines of text formatted as capital letters only, with not much spacing between letters, and line spacing the same height as the lines themselves, effectively turning it into high contrast evenly alternating rows

Use an automated system to check gameplay for stimuli that could trigger photosensitive epilepsy. (Example: The Harding Test and Harding Flash and Pattern Analyzer (FPA) G2 developed by Cambridge Research System Ltd and Professor Graham Harding.)

Include **Flashing On/Off** as a setting option and set **Flashing** as **Off** by default. In doing so, you protect players who do not yet know they are susceptible to seizures.

Design for breaks between game levels, encouraging players to take a break from playing non-stop.

Other accessibility resources

Here are some external sites that provide additional information about game accessibility.

Game accessibility guidelines

- Game accessibility guidelines (used as a reference in this topic)
- AbleGamers Foundation guidelines (used as a reference in this topic)
- Design Universally Accessible (UA) games

Custom input controllers

- SpecialEffect
- War fighter engaged

Other references used

- Color Blind Awareness, a Community Interest Company
- How to do subtitles well—a blog article on Gamasutra by Ian Hamilton
- Innovation for All Programme
- Epilepsy foundation

Related links

- Inclusive Design
- Accessibility in Windows 11 and Windows 10
- Developing accessible UWP apps
- Engineering Software For Accessibility eBook

Using cloud services for UWP games

Article • 03/16/2023

The Universal Windows Platform (UWP) in Windows 10 offers a set of APIs that can be used for developing games across Microsoft devices. When developing games across platforms and devices, you can make use of a cloud backend to help scale your game according to demand.

If you are looking for a complete cloud backend solution for your game, see Software as a Service for game backend.

What is cloud computing?

Cloud computing uses on demand IT resources and applications over the internet to store and process data for your devices. The term *cloud* is a metaphor for the availability of vast resources out there (not local resources) that you can access from non-specific locations. The principle of cloud computing offers a new way in which resources and software can be consumed. Users no longer need to pay for the full complete product or resources upfront, but instead are able to consume platform, software, and resources as a service. Cloud providers often bill their customers according to usage or service plan offerings.

Why use cloud services?

One advantage of using cloud services for games is that you do not need to invest in physical hardware servers upfront, but only need to pay according to usage or service plans at a later stage. It is one way to help manage the risks involved in developing a new game title.

Another advantage is that your game can tap into vast cloud resources to achieve scalability (effectively manage any sudden spikes in the number of concurrent players, intense real-time game calculations or data requirements). This keeps the performance of your game stable around the clock. Furthermore, cloud resources can be accessed from any device running on any platform anywhere in the world, which means that you are able to bring your game to everyone globally.

Delivering an amazing gameplay experience to your players is important. Because game servers running in the cloud are independent of client-side updates, they can give you a more controlled and secure environment for your game overall. You can also achieve gameplay consistency through the cloud by never trusting the client and having server

side game logic. Service-to-service connections can also be configured to allow a more integrated gaming experience; examples include linking in-game purchases to various payment methods, bridging over different gaming networks, and sharing in-game updates to popular social media portals such as Facebook and X.

You can also use dedicated cloud servers to create a large persistent game world, build up a gamer community, collect and analyze gamer data over time to improve gameplay, and optimize your game's monetization design model.

In addition, games that require intensive game data management capabilities like social games with asynchronous multiplayer mechanics can be implemented using cloud services.

How game companies use the cloud technology

Learn how other developers have implemented cloud solutions in their games.

Expand table

Developer	Description	Key game scenarios	Learn more
Tencent Games	**Tencent Games** has a developed an innovative solution using Azure Service Fabric enabling traditional PC games to be delivered as a service. Their Cloud Game Solution uses a 'thin client + rich cloud' model running workloads as microservices in the backend.	• Traditional PC games are delivered as cloud games to users around the world • Optimized game delivery process • Game functionalities isolated as microservices to reduce complexity, reduce workloads repetition due to dependencies, and ability to upgrade new features independently • Small installation package downloads to play newest game content (Reduced package size from GB to MB)	• Tencent Games and Microsoft built the cloud game solution • Building Games with Service Fabric: Details about Tencent's implementation (video)

		• Reduced maintenance cost	
343 Industries	**Halo 5: Guardians** implemented Halo: Spartan Companies as its social gameplay platform by using Azure Cosmos DB (via DocumentDB API), which was selected for its speed and flexibility due to its auto-indexing capabilities.	• Scalable data-tier to handle groups creation/management for multiplayer gameplay • Game and social media integration • Real-time queries of data through multiple attributes • Synchronization of gameplay achievements and stats	
Illyriad Games	Illyriad Games created **Age of Ascent**, a massively multiplayer online (MMO) epic 3D space game that can be played on devices that have modern browsers. So this game can be played on PCs, laptops, mobile phones and other mobile devices without plug-ins. The game uses ASP.NET Core, HTML5, WebGL, and Azure.	• Cross-platform, browser-based game • Single large persistent open world • Handles intensive real-time gameplay calculations • Scales with number of players	• Building games with Service Fabric: Age of Ascent MMO game (video) • Manage game components as microservices using Azure Service Fabric (video) • Interview with Age of Ascent developers (video)
Next Games	Next Games is the creator of **The Walking Dead: No Man's Land** video game which is based on AMC's original series. The Walking Dead game used Azure as the backend. It had 1,000,000 downloads in the opening weekend and within the first week, the game became #1 iPhone & iPad Free App in the	• Cross-platform • Turn based multiplayer • Elastically scale performance • Gamer fraud protection • Dynamic content delivery	• How we built it: Next Games global online gaming platform on Azure (blog with video)

	U.S. App Store, #1 Free App in 12 countries, and #1 Free Game in 13 countries.		
Pixel Squad	Pixel Squad developed **Crime Coast** using Unity game engine and Azure. **Crime Coast** is a social strategy game available on the Android, iOS and Windows platform. Azure Blob Storage, Managed Azure Redis Cache, an array of load balanced IIS VMs, and Microsoft Notification hub were used in their game. Learn how they managed scaling and handled players surge with 5000 simultaneous players.	• Cross-platform • Multiplayer online game • Scale with number of players	• How Crime Coast MMO game used Azure Cloud Services

Other links

- Hitman and Azure: Create game features like Elusive Target that are only possible using cloud
- Azure as the secret sauce for Hitcents, Game Troopers and InnoSpark

How to design your cloud backend

While producers and game designers are in discussion about what game features and functionalities are needed in the game, it is good to start considering how you want to design your game infrastructure. Azure can be used as your game backend when you want to develop games for various devices and across different major platforms.

Understanding IaaS, PaaS or SaaS

First, you need to think about the level of service that is best suited for your game. Knowing the differences in the following three services can help you determine the approach you want to take in building your backend.

- Infrastructure as a Service (IaaS)

Infrastructure as a Service (IaaS) is an instant computing infrastructure, provisioned and managed over the Internet. Imagine having the possibility of many machines readily available to quickly scale up and down depending on demand. IaaS helps you to avoid the cost and complexity of buying and managing your own physical servers and other datacenter infrastructure.

- Platform as a Service (PaaS)

  Platform as a Service (PaaS) is like IaaS but it also includes management of infrastructure like servers, storage, and networking. So on the top of not buying physical servers and datacenter infrastructure, you also do not need to buy and manage software licenses, underlying application infrastructure, middleware, development tools, or other resources.

- Software as a Service (SaaS)

  Software as a service (SaaS) allows users to connect to and use cloud-based apps over the Internet. It provides a complete software solution that you purchase on a pay-as-you-go basis from a cloud service provider. Common examples are email, calendaring, and office tools (such as Microsoft 365 Office apps). You rent the use of an app for your organization, and your users connect to it over the Internet, usually with a web browser. All of the underlying infrastructure, middleware, app software, and app data are located in the service provider's data center. The service provider manages the hardware and software, and with the appropriate service agreement, will ensure the availability and the security of the game and your data as well. SaaS allows your organization to get quickly up and running with an app at minimal upfront cost.

Design your game infrastructure using Azure

Following are some ways that Azure cloud offerings can be used for a game. Azure works with Windows, Linux, and familiar open source technologies such as Ruby, Python, Java, and PHP. For more information, see Azure for gaming.

Expand table

Requirements	Activity scenarios	Product Offering	Product Capabilities
Host your domain in the cloud	Respond to DNS queries efficiently	Azure DNS	Host your domain with high performance and availability

Requirements	Activity scenarios	Product Offering	Product Capabilities
Sign in, identity verification	Gamer signs in and gamer identity is authenticated	Azure Active Directory	Single sign-on to any cloud and on-premises web app with multi-factor authentication
Game using infrastructure as a service model (IaaS)	Game is hosted on virtual machines in the cloud	Azure VMs	Scale from 1 to thousands of virtual machine instances as game servers with built-in virtual networking and load balancing; hybrid consistency with on-premises systems
Web or mobile games using platform as a service model (PaaS)	Game is hosted on a managed platform	Azure App Service	PaaS for websites or mobile games (which means Azure VMs with middleware/development tools/BI/DB management)
Highly available, scalable n-tier cloud game with more control of the OS (PaaS)	Game is hosted on a managed platform	Azure Cloud Service	PaaS designed to support applications that are scalable, reliable, and cheap to operate
Load balancing across regions for better performance and availability	Routes incoming game requests. Can act as first level of load balancing.	Azure Traffic Manager	Offers multiple automatic failover options and ability to distribute your traffic equally or with weighted values. Can seamlessly combine on-premises and cloud systems.
Cloud storage for game data	Latest game data is stored in the cloud and sent to client devices	Azure Blob Storage	No restriction on the kinds of file that can be stored; object storage for large amounts of unstructured data like images, audio, video, and more.
Temporary data storage tables	Game transactions (changes in game states) are stored in tables temporarily	Azure Table Storage	Game data can be stored in a flexible schema according to the needs of the game
Queue game transactions/requests	Game transactions are processed in the form of a queue	Azure Queue Storage	Queues absorb unexpected traffic bursts and can prevent servers from being overwhelmed

Requirements	Activity scenarios	Product Offering	Product Capabilities
			by a sudden flood of requests during the game
Scalable relational game database	Structured storage of relational data like in-game transactions to database	Azure SQL Database	SQL database as a service (Compare with SQL on a VM)
Scalable distributed low-latency game database	Fast read, write, and query of game and player data with schema flexibility	Azure Cosmos DB	Low latency NoSQL document database as a service
Use own datacenter with Azure services	Game is retrieved from your own datacenter and sent to the client devices	Azure Stack	Enables your organization to deliver Azure services from your own datacenter to help you achieve more
Large data chunks transfer	Large files such as game images, audio, and videos can be sent to users from the nearest Content Delivery Network (CDN) pop location with Azure CDN	Azure Content Delivery Network	Built on a modern network topology of large centralized nodes, Azure CDN handles sudden traffic spikes and heavy loads to dramatically increase speed and availability, resulting in significant user experience improvements
Low latency	Perform caching to build fast, scalable games with more control and guaranteed isolation of data; can be used to improve match-making feature for game as well.	Azure Redis Cache	High throughput, consistent low-latency data access to power fast, scalable Azure applications

Requirements	Activity scenarios	Product Offering	Product Capabilities
High scalability, low latency	Handles fluctuations in the number of game users with low latency read and writes	Azure Service Fabric	Able to power the most complex, low-latency, data-intensive scenarios and reliably scale to handle more users at a time. Service Fabric enables you to build games without having to create a separate store or cache, as required for stateless apps
Ability to collect millions of events per second from devices	Log millions of events per second from devices	Azure Event Hubs	Cloud-scale telemetry ingestion from games, websites, apps, and devices
Real time processing for game data	Perform real-time analysis of gamer data to improve gameplay	Azure Stream Analytics	Real-time stream processing in the cloud
Develop predictive gameplay	Create customized dynamic gameplay based on gamer data	Azure Machine Learning	A fully managed cloud service that enables you to easily build, deploy, and share predictive analytics solutions
Collect and analyze game data	Massive parallel processing of data from both relational and non-relational databases	Azure Data Warehouse	Elastic data warehouse as a service with Enterprise class features
Engage users to increase usage and retention	Send targeted push notifications to any platform from any back end to generate interest and encourage specific game actions	Azure Notification Hubs	Fast broadcast push to reach millions of mobile devices on all major platforms — iOS, Android, Windows, Kindle, Baidu. Your game can be hosted on any back end — cloud or on-premises.
Stream media content to your local and	Broadcast quality game trailers	Azure Media Services	On-demand and live video streaming with integrated

Requirements	Activity scenarios	Product Offering	Product Capabilities
worldwide audiences while protecting your content	and cinematic clips can be watched from all devices		Content Delivery Network capabilities. Use one player for all of your playback needs, includes content protection and encryption.
Develop, distribute, and beta-test your mobile apps	Test and distribute your mobile app. App performance and user experience management.	HockeyApp	Integrates crash reporting and user metrics with an app distribution and user feedback platform. Supports Android, Cordova, iOS, OS X, Unity, Windows, and Xamarin apps. Also, consider Visual Studio Mobile Center — mission control for apps that combines rich analytics, crash reporting, push notifications, app distribution, and more.
Create marketing campaigns to increase usage and retention	Send push notifications to targeted players to generate interest and encourage specific game actions according to data analysis	Mobile engagement - will be retired March 2018 and is currently only available to existing customers	Increase gameplay time and user retention on all major platforms —iOS, Android, Windows, Windows Phone

Startup and developer resources

- Microsoft for Startups

  Microsoft for Startups provides product, technical, and go-to-market benefits to help accelerate the growth of startups. One benefit includes getting an Azure free account. You have $200 credit to explore services for 30 days, 12 months of popular free services, and always free 25+ services. For more information, see Bring your startup's ideas to life with an Azure free account.

- Developer programs

  Microsoft offers several developer programs like ID@Xbox and Xbox Live Creators Program to help you develop and publish games.

Learning resources

- //build 2016: CodeLabs — Use Microsoft Azure App Service and Microsoft SQL Azure backend to save game score in Unity
- //build 2017: Delivering world-class game experiences using Microsoft Azure: Lessons learned from titles like Halo, Hitman, and Walking Dead (video)
- Reusable set of building blocks, projects, services, and best practices designed to support common gaming workloads using Azure on GitHub: Building blocks for gaming on Azure
- Gaming Services on Azure (videos)

Tools and other useful links

- MSDN forums — Azure platform
- Cloud Based Load Testing tool
- SDKs and command-line tools

Software as a Service for game backend

Azure PlayFab currently powers more than 1,200 live games with 80 million monthly active players. It's a complete backend platform that includes full stack LiveOps with real-time control.

You can integrate this solution into your mobile, PC, or console games using SDKs. There are SDKs available for all popular game engines and platforms, including Android, iOS, Unreal, Unity, and Windows.

It offers game services like authentication, player data management, multiplayer, and real-time analytics to help your game grow its user base. Harness the power of real-time data pipeline and LiveOps to engage your users with customized in-game items, events, and promotions. You also have the ability to conduct A/B testing, generate reports, send push notifications, and more.

We're constantly innovating and adding new features. For more information, see Azure PlayFab; and for pricing, see Pricing.

Related links

- Windows game development guide
- Azure for gaming
- Azure PlayFab

- Microsoft for Startups
- ID@Xbox

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

Monetization for games

Article • 03/16/2023

As a game developer, you need to know your monetization options so you can sustain your business and keep doing what you're passionate about: creating great games. This article provides an overview of the monetization methods for a Universal Windows Platform (UWP) game and how to implement them.

In the past, you would simply put a price on your game and then wait for people to purchase it at a store. But today you have options. You can choose to distribute a game to "brick-and-mortar" stores, sell the game online (either physical or soft copies), or let everyone play the game for free but incorporate some sort of ads or in-game items that can be purchased. Games are also no longer just standalone products. They often come with extra content that can be purchased in addition to the main game.

You can promote and monetize a UWP game in one or more of these ways:

- Put your game in the Microsoft Store, which is a secured, online store offering worldwide distribution. Gamers around the world can buy your game online at the price you set.
- Use APIs in the Windows SDK to create in-game purchases. Gamers can buy items from within your game, or buy additional content such as extra equipment, skins, maps, or game levels.
- Use APIs in the Microsoft Advertising SDK to display ads from ad networks. You can display ads in your game and offer the option for gamers to watch video ads in exchange for in-game rewards.
- Maximize your game's potential through ad campaigns. Promote your game using paid, community (free), or house (free) ad campaigns to grow its user base.

Worldwide distribution channel

The Microsoft Store can make your game available for download in more than 200 countries and regions worldwide, with support for billing via various forms of payment including Visa, MasterCard, and PayPal. For a full list of countries and regions, see Define market selection.

Set a price for your game

UWP games published to the Store can be either be *paid* or *free*. A paid game allows you to charge gamers up front for your game at a price you set, whereas a free game

allows users to download and play the game without paying for it.

Here are some important concepts regarding the pricing of your game in the Store.

Base price

The base price of the game is what determines whether your game is categorized as *paid* or *free*. You can use Partner Center to configure the base price based on country and region. The process of determining the price may include your tax responsibilities when selling to different countries and cost considerations for specific markets. You can also set custom prices for specific markets.

Sale price

One way to promote your game is to reduce its price for a limited time. It's also possible to set the sale price to **Free** to allow your game to be downloaded without payment. You can schedule sale campaigns in advance by setting both the starting date and ending date of the sale. For more info, see Put apps and add-ons on sale.

In-game purchases

In-game purchases are products bought within a game. They're also generically known as *in-app purchases*. In the Microsoft Store, these products are called *add-ons*. Add-ons are published through Partner Center. You'll also need to enable the add-ons in your game's code.

Types of add-ons

You can create two types of add-ons in the store: *durables* or *consumables*. Durables are items that persist over for a specified amount of time and can be purchased only once until they expire. Consumables are items that can be purchased and used again and again.

When creating consumables, decide how you want to keep track of them — that is whether they're *developer managed* or *Store managed* (This feature is available starting in Windows 10, version 1607). With a developer-managed consumable, you are responsible for keeping track of the item's balance for the gamer; with a Store-managed consumable, the Microsoft Store keeps track of the item's balance for you. For more info, see Overview of consumable add-ons.

Create in-game purchases

The latest in-app purchases and license info APIs are part of the Windows.Services.Store namespace in the Windows SDK (starting in Windows 10, version 1607). If you're developing a new game that targets 1607 or later release, we recommend that you use the **Windows.Services.Store** namespace because it supports the latest add-on types and has better performance. It's also designed to be compatible with future types of products and features supported by the Partner Center and the Store. When developing for previous versions of Windows 10, use the Windows.ApplicationModel.Store namespace instead.

For more info, go to In-app purchases and trials.

Simplified purchase example

This section uses a simplified purchase example to illustrate the use of different method calls to implement the purchase flow.

In-game actions / activity	**Game background tasks**
Gamer enters a shop. Shop menu pops up to display the available add-ons and purchase price	Game retrieves the product info of the add-ons, determines whether the add-ons have the proper license, and displays the add-ons that are available for purchase by the gamer on the shop menu.
Gamer clicks **Buy** to purchase an item	The **Buy** action sends a request to purchase the item and starts the payment process to acquire it. The implementation varies depending on the item type. If it is a durable or a one-time purchase item, the customer can own only a single item until it expires. If the item is a consumable, the customer can own one or more of it.

Test in-game purchases during game development

Because an add-on must be created in association with a game, your game must be published and available in the Store. The steps in this section show how to create add-ons while your game is still in development. (If your finished game is already live in the Store, you can skip the first three steps and go directly to Create an add-on in the Store.)

To create add-ons while your game is still in development:

1. Create a package
2. Publish the game as hidden

3. Associate your game solution in Visual Studio with the Store
4. Create an add-on in the Store

Create a package

For any game to be published, it must meet the minimum Windows App Certification requirements. You can use the Windows App Certification Kit, which is part of the Windows SDK, to run tests on the game to help ensure that it's ready for publishing to the Store. If you haven't already downloaded the Windows SDK that includes the Windows App Certification Kit, then go to Windows SDK.

To create a package that can be uploaded to the Store:

1. Open your game solution in Visual Studio.
2. Within Visual Studio, go to **Project** > **Store** > **Create App Packages ...**
3. For the **Do you want to build packages to upload to the Microsoft Store?** option, select **Yes**.
4. Sign in to your Partner Center developer account. Or register as a developer in Partner Center.
5. Select an app to create the upload package for. If you have not yet created an app submission, provide a new app name to create a new submission. For more info, see Create your app by reserving a name.
6. After the package has been created successfully, click **Launch Windows App Certification Kit** to start the testing process.
7. Fix any errors to create a game package.

Publish the game as hidden

1. Go to Partner Center and sign in.
2. From the **Dashboard overview** or **All apps** page, click the app you want to work with. If you have not yet created an app submission, click on **Create a new app** and reserve a name.
3. On the **App Overview** page, click **Start your submission**.
4. Configure this new submission. On the submission page:

    - Click **Pricing and availability**. In the **Visibility** section, choose '**Hide this app and prevent acquisition...**' to ensure only your development team has access to the game. For more details, go to Distribution and visibility.
    - Click **Properties**. In the **Category and subcategory** section, choose **Games** and then a suitable subcategory for your game.
    - Click **Age ratings**. Fill out the questionnaire accurately.

    - Click **Packages**. Upload the game package created in the earlier step.
5. Follow any other submission prompts in the dashboard to allow you to successfully publish this game which remains hidden to the public.
6. Click **Submit to the Store**.

For more info, go to App submissions.

After your game is submitted to the Store, it enters the app certification process. This process can take up to 16 hours before the game is listed.

Associate your game solution with the Store

With your game solution opened in Visual Studio:

1. Go to **Project** > **Store** > **Associate App with the Store ...**
2. Sign in to your Partner Center developer account and select the app name to associate this solution with.
3. Double-click on the **Package.appxmanifest.xml file** and go to the **Packaging** tab to check that the game is associated correctly.

If you have associated the solution to a published game that is live and listed in the Store, your solution will have an active license and you are one step closer in creating add-ons for your game. For more info, see Packaging apps.

Create an add-on in the Store

As you create add-ons, make sure you're associating them with the right game submission. For details about how to configure all the various info associated with an add-on, see Add-on submissions.

1. Go to Partner Center and sign in.
2. From the **Dashboard overview** or **All apps** page, click the app you want to create the add-on for.
3. On the **App Overview** page, in the **Add-ons** section, select **Create a new add-on**.
4. Select the product type for the add-on: **developer-managed consumable**, **store-managed consumable**, or **durable**.
5. Enter a unique product ID which will be used as a string variable when integrating this add-on into your game code. This ID will not be seen by consumers. For more info, see Set your app product type and product ID.

Other configurations for add-ons include:

- Properties

- Pricing and availability
- Store listing

If your game has many add-ons, you can create them programmatically by using the **Microsoft Store submission API**. For more info, see Create and manage submissions using Microsoft Store services.

Display ads in your game

The libraries and tools in the Microsoft Advertising SDK help you set up a service in your game to receive ads from an ad network. Your gamers will be shown live ads and you'll earn money from the advertisers when your gamers view or interact with the displayed ads. For more info, see Display ads in your app.

Ad formats

Several types of ads can be displayed by using the Microsoft Advertising SDK:

- Banner ads — Ads that take up a part of your gaming screen and are usually placed within a game.
- Interstitial video ads — Full-screen ads, which can be very effective when used between levels. If implemented properly, they can be less obtrusive than banner ads.
- Native ads — Component-based ads, where each piece of the ad creative (such as the title, image, description, and call-to-action text) is delivered to your app as an individual element that you can integrate into your app.

Which ads are displayed?

By default, your app will show ads from Microsoft's network for paid ads. To maximize your ad revenue, you can enable ad mediation for your ad unit to display ads from additional paid ad networks. For more info about current offerings, see our Mediation settings guidance.

Which markets allow ads to be displayed?

For the full list of countries and regions that support ads, see Supported markets for ad networks.

APIs for displaying ads

The AdControl, InterstitialAd, and NativeAd classes in the Microsoft Advertising SDK are used to help display ads in games.

To get started, download and install the Microsoft Advertising SDK with Visual Studio 2015 or a later version. For more info, see Install the Microsoft Advertising SDK.

Implementation guides

These walkthroughs show how to implement ads by using **AdControl**, **InterstitialAd**, and **NativeAd**:

- Create banner ads in XAML and .NET
- Create banner ads in HTML5 and JavaScript
- Create interstitial ads
- Create native ads

During development, you can make use of the test ad unit values to see how the ads are rendered. These test ad unit values are also used in the walkthroughs above.

Here are some best practices to help you in the design and implementation process.

- Best practices for banner ads
- Best practices for interstitial ads

For solutions to common development issues, like ads not appearing, black box blinking and disappearing, or ads not refreshing, see Troubleshooting guides.

Prepare for release by replacing ad unit test values

When you're ready to move to live testing or to receive ads in published games, you must update the test ad unit values to the actual values provided for your game. To create ad units for your game, see Set up ad units in your app.

Other ad networks

These are other ad networks that provide SDKs for serving ads to UWP apps and games.

Vungle

The Vungle SDK for Windows offer video ads in apps and games. To download the SDK, go to Vungle SDK.

Smaato

Smaato enables banner ads to be incorporated into UWP apps and games. Download the SDK and for more info, see the documentation.

AdDuplex

You can use AdDuplex to implement banner or interstitial ads in your game.

To learn more about integrating AdDuplex directly into a Windows 10 XAML project, go to the AdDuplex website:

- Banner ads: Windows 10 SDK for XAML
- Interstitial ads: Windows 10 XAML AdDuplex Interstitial Ad Installation and Usage

For info about integrating the AdDuplex SDK into Windows 10 UWP games created using Unity, see Windows 10 SDK for Unity apps installation and usage.

Maximize your game's potential through ad campaigns

Take the next step in promoting your game using ads. When you create an ad campaign for your game, other apps and games will display ads promoting your game.

Choose from several types of campaigns that can help increase your gamer base.

Campaign type	Ads for your game appear in...
Paid	Apps that match your game's device or category.
Free community	Apps published by other developers who have also opted in to community ad campaigns. For more info, see About community ads.
Free house	Only apps that you've published. For more info, see About house ads.

Related links

- Getting paid
- Account types, locations, and fees
- Analytics
- Globalization and localization
- Implement a trial version of your app

- Run app experiments with A/B testing

Concept approval

Article • 04/04/2023

Concept approval is the process of submitting a proposal for a game to Microsoft. This up-front, high-level submission benefits both Microsoft and you by identifying at the very beginning of the process any likely difficulties or drawbacks in the overall plan for the game. Try to make sure that your content isn't overly vulgar, offensive, or objectionable, and that it feels at home on the target platform. Once you submit your proposal, Microsoft will review it and then notify you of the result.

Who needs concept approval?

This process is only required if you are publishing a game to Xbox through ID@Xbox or as a managed partner. You don't need to go through this process if you join the Xbox Live Creators Program and make a Universal Windows Platform (UWP) game, which you can then self-publish to Xbox. However, games made through this program will be featured in a separate section of the Store. If you want your game to be featured alongside big AAA games, or if you want to create a more intensive game using the Xbox Development Kit (XDK), you'll need to go through concept approval.

You also don't need concept approval if you're developing a UWP game for Windows desktop or mobile devices (or if you're publishing a UWP app that's *not* a game, targeting *any* device). All you need is to sign up as a developer, and you can freely configure and submit your app to the Store through Partner Center.

Submit your concept for approval

If you are an independent game developer or publisher, you can submit your concept for approval through the ID@Xbox program. Learn more about ID@Xbox and apply here.

If you are already an ID@Xbox developer, you should have been sent a link to the Game Information Form (GIF) where you can submit your game concept. If you have questions, contact id@xbox.com.

If you have an existing license agreement with Microsoft, contact your Microsoft account team for information on submitting your concept.

Package your Universal Windows Platform (UWP) DirectX game

Article • 10/27/2022

Larger Universal Windows Platform (UWP) games, especially those that support multiple languages with region-specific assets or feature optional high-definition assets, can easily balloon to large sizes. In this topic, learn how to use app packages and app bundles to customize your app so that your customers only receive the resources they actually need.

In addition to the app package model, Windows 10 supports app bundles which group together two types of packs:

- App packs contain platform-specific executables and libraries. Typically, a UWP game can have up to three app packs: one each for the x86, x64, and Arm CPU architectures. All code and data specific to that hardware platform must be included in its app pack. An app pack should also contain all the core assets for the game to run with a baseline level of fidelity and performance.
- Resource packs contain optional or expanded platform-agnostic data, such as game assets (textures, meshes, sound, text). A UWP game can have one or more resource packs, including resource packs for high-definition assets or textures, DirectX feature level 11+ resources, or language-specific assets and resources.

For more information about app bundles and app packs, read Defining app resources.

While you can place all content in your app packs, this is inefficient and redundant. Why have the same large texture file replicated three times for each platform, especially for Arm platforms that may not use it? A good goal is to try to minimize what your customer has to download, so they can start playing your game sooner, save space on their device, and avoid possible metered bandwidth costs.

To use this feature of the UWP app installer, it is important to consider the directory layout and file naming conventions for app and resource packaging early in game development, so your tools and source can output them correctly in a way that makes packaging simple. Follow the rules outlined in this doc when developing or configuring asset creation and managing tools and scripts, and when authoring code that loads or references resources.

Why create resource packs?

When you create an app, particularly a game app that can be sold in many locales or a broad variety of UWP hardware platforms, you often need to include multiple versions of many files to support those locales or platforms. For example, if you are releasing your game in both the United States and Japan, you might need one set of voice files in English for the en-us locales, and another in Japanese for the jp-jp locale. Or, if you want to use an image in your game for Arm devices as well as x86 and x64 platforms, you must upload the same image asset 3 times, once for each CPU architecture.

Additionally, if your game has a lot of high definition resources that do not apply to platforms with lower DirectX feature levels, why include them in the baseline app pack and require your user to a download a large volume of components that the device can't use? Separating these high-def resources into an optional resource pack means that customers with devices that support those high-def resources can obtain them at the cost of (possibly metered) bandwidth, while those who do not have higher-end devices can get their game quicker and at a lower network usage cost.

Content candidates for game resource packs include:

- International locale specific assets (localized text, audio, or images)
- High resolution assets for different device scaling factors (1.0x, 1.4x, and 1.8x)
- High definition assets for higher DirectX feature levels (9, 10, and 11)

All of this is defined in the package.appxmanifest that is part of your UWP project, and in your directory structure of your final package. Because of the new Visual Studio UI, if you follow the process in this document, you should not need to edit it manually.

> **Important** The loading and management of these resources are handled through the **Windows.ApplicationModel.Resources*** APIs. If you use these app model resource APIs to load the correct file for a locale, scaling factor, or DirectX feature level, you do not need to load your assets using explicit file paths; rather, you provide the resource APIs with just the generalized file name of the asset you want, and let the resource management system obtain the correct variant of the resource for the user's current platform and locale configuration (which you can specify directly as well with these same APIs).

Resources for resource packaging are specified in one of two basic ways:

- Asset files have the same filename, and the resource pack specific versions are placed in specific named directories. These directory names are reserved by the system. For example, \en-us, \scale-140, \dxfl-dx11.
- Asset files are stored in folders with arbitrary names, but the files are named with a common label that is appended with strings reserved by the system to denote

language or other qualifiers. Specifically, the qualifier strings are affixed to the generalized filename after an underscore (“_”). For example, \assets\menu_option1_lang-en-us.png, \assets\menu_option1_scale-140.png, \assets\coolsign_dxfl-dx11.dds. You may also combine these strings. For example, \assets\menu_option1_scale-140_lang-en-us.png.

> **Note** When used in a filename rather than alone in a directory name, a language qualifier must take the form "lang-<tag>", for example, "lang-en-us" as described in Tailor your resources for language, scale, and other qualifiers.

Directory names can be combined for additional specificity in resource packaging. However, they cannot be redundant. For example, \en-us\menu_option1_lang-en-us.png is redundant.

You may specify any non-reserved subdirectory names you need underneath a resource directory, as long as the directory structure is identical in each resource directory. For example, \dxfl-dx10\assets\textures\coolsign.dds. When you load or reference an asset, the pathname must be generalized, removing any qualifiers for language, scale, or DirectX feature level, whether they are in folder nodes or in the file names. For example, to refer in code to an asset for which one of the variants is \dxfl-dx10\assets\textures\coolsign.dds, use \assets\textures\coolsign.dds. Likewise, to refer to an asset with a variant \images\background_scale-140.png, use \images\background.png.

Here are the following reserved directory names and filename underscore prefixes:

Asset type	Resource pack directory name	Resource pack filename suffix
Localized assets	All possible languages, or language and locale combinations, for Windows 10. (The qualifier prefix "lang-" is not required in a folder name.)	An "_" followed by the language, locale, or language-locale specifier. For example, "_en", "_us", or "_en-us", respectively.
Scaling factor assets	scale-100, scale-140, scale-180. These are for the 1.0x, 1.4x, and 1.8x UI scaling factors, respectively.	An "_" followed by "scale-100", "scale-140", or "scale-180".
DirectX feature level assets	dxfl-dx9, dxfl-dx10, and dxfl-dx11. These are for the DirectX 9, 10, and 11 feature levels, respectively.	An "_" followed by "dxfl-dx9", "dxfl-dx10", or "dxfl-dx11".

Defining localized language resource packs

Locale-specific files are placed in project directories named for the language (for example, "en").

When configuring your app to support localized assets for multiple languages, you should:

- Create an app subdirectory (or file version) for each language and locale you will support (for example, en-us, jp-jp, zh-cn, fr-fr, and so on).
- During development, place copies of ALL assets (such as localized audio files, textures, and menu graphics) in the corresponding language locale subdirectory, even if they are not different across languages or locales. For the best user experience, ensure that the user is alerted if they have not obtained an available language resource pack for their locale if one is available (or if they have accidentally deleted it after download and installation).
- Make sure each asset or string resource file (.resw) has the same name in each directory. For example, menu_option1.png should have the same name in both the \en-us and \jp-jp directories even if the content of the file is for a different language. In this case, you'd see them as \en-us\menu_option1.png and \jp-jp\menu_option1.png.

  > **Note** You can optionally append the locale to the file name and store them in the same directory; for example, \assets\menu_option1_lang-en-us.png, \assets\menu_option1_lang-jp-jp.png.

- Use the APIs in **Windows.ApplicationModel.Resources** and **Windows.ApplicationModel.Resources.Core** to specify and load the locale-specific resources for you app. Also, use asset references that do no include the specific locale, since these APIs determine the correct locale based on the user's settings and then retrieve the correct resource for the user.
- In Microsoft Visual Studio 2015, select **PROJECT->Store->Create App Package...** and create the package.

Defining scaling factor resource packs

Windows 10 provides three user interface scaling factors: 1.0x, 1.4x, and 1.8x. Scaling values for each display are set during installation based on a number of combined factors: the size of the screen, the resolution of the screen, and the assumed average distance of the user from the screen. The user can also adjust scale factors to improve readability. Your game should be both DPI-aware and scaling factor-aware for the best possible experience. Part of this awareness means creating versions of critical visual assets for each of the three scaling factors. This also includes pointer interaction and hit testing!

When configuring your app to support resource packs for different UWP app scaling factors, you should:

- Create an app subdirectory (or file version) for each scaling factor you will support (scale-100, scale-140, and scale-180).
- During development, place scale factor-appropriate copies of ALL assets in each scale factor resource directory, even if they are not different across scaling factors.
- Make sure each asset has the same name in each directory. For example, menu_option1.png should have the same name in both the \scale-100 and \scale-180 directories even if the content of the file is different. In this case, you'd see them as \scale-100\menu_option1.png and \scale-140\menu_option1.png.

  > **Note** Again, you can optionally append the scaling factor suffix to the file name and store them in the same directory; for example, \assets\menu_option1_scale-100.png, \assets\menu_option1_scale-140.png.

- Use the APIs in **Windows.ApplicationModel.Resources.Core** to load the assets. Asset references should be generalized (no suffix), leaving out the specific scale variation. The system will retrieve the appropriate scale asset for the display and the user's settings.
- In Visual Studio 2015, select **PROJECT->Store->Create App Package...** and create the package.

Defining DirectX feature level resource packs

DirectX feature levels correspond to GPU feature sets for prior and current versions of DirectX (specifically, Direct3D). This includes shader model specifications and

functionality, shader language support, texture compression support, and overall graphics pipeline features.

Your baseline app pack should use the baseline texture compression formats: BC1, BC2, or BC3. These formats can be consumed by any UWP device, from low-end Arm platforms up to dedicated multi-GPU workstations and media computers.

Texture format support at DirectX feature level 10 or later should be added in a resource pack to conserve local disk space and download bandwidth. This enables using the more advanced compression schemes for 11, like BC6H and BC7. (For more details, see Texture block compression in Direct3D 11.) These formats are more efficient for the high-resolution texture assets supported by modern GPUs, and using them improves the look, performance, and space requirements of your game on high-end platforms.

DirectX feature level	Supported texture compression
9	BC1, BC2, BC3
10	BC4, BC5
11	BC6H, BC7

Also, each DirectX feature levels supports different shader model versions. Compiled shader resources can be created on a per-feature level basis, and can be included in DirectX feature level resource packs. Additionally, some later version shader models can use assets, such as normal maps, that earlier shader model versions cannot. These shader model specific assets should be included in a DirectX feature level resource pack as well.

The resource mechanism is primarily focused on texture formats supported for assets, so it supports only the 3 overall feature levels. If you need to have separate shaders for sub-levels (dot versions) like DX9_1 vs DX9_3, your asset management and rendering code must handle them explicitly.

When configuring your app to support resource packs for different DirectX feature levels, you should:

- Create an app subdirectory (or file version) for each DirectX feature level you will support (dxfl-dx9, dxfl-dx10, and dxfl-dx11).
- During development, place feature level specific assets in each feature level resource directory. Unlike locales and scaling factors, you may have different rendering code branches for each feature level in your game, and if you have

textures, compiled shaders, or other assets that are only used in one or a subset of all supported feature levels, put the corresponding assets only in the directories for the feature levels that use them. For assets that are loaded across all feature levels, make sure that each feature level resource directory has a version of it with the same name. For example, for a feature level independent texture named "coolsign.dds", place the BC3-compressed version in the \dxfl-dx9 directory and the BC7-compressed version in the \dxfl-dx11 directory.

- Make sure each asset (if it is available to multiple feature levels) has the same name in each directory. For example, coolsign.dds should have the same name in both the \dxfl-dx9 and \dxfl-dx11 directories even if the content of the file is different. In this case, you'd see them as \dxfl-dx9\coolsign.dds and \dxfl-dx11\coolsign.dds.

  > **Note** Again, you can optionally append the feature level suffix to the file name and store them in the same directory; for example, \textures\coolsign_dxfl-dx9.dds, \textures\coolsign_dxfl-dx11.dds.

- Declare the supported DirectX feature levels when configuring your graphics resources.

```
C++

D3D_FEATURE_LEVEL featureLevels[] =
{
  D3D_FEATURE_LEVEL_11_1,
  D3D_FEATURE_LEVEL_11_0,
  D3D_FEATURE_LEVEL_10_1,
  D3D_FEATURE_LEVEL_10_0,
  D3D_FEATURE_LEVEL_9_3,
  D3D_FEATURE_LEVEL_9_1
};
```

```
C++

ComPtr<ID3D11Device> device;
ComPtr<ID3D11DeviceContext> context;
D3D11CreateDevice(
    nullptr,                    // Use the default adapter.
    D3D_DRIVER_TYPE_HARDWARE,
    0,                      // Use 0 unless it is a software device.
    creationFlags,          // defined above
    featureLevels,          // What the app will support.
    ARRAYSIZE(featureLevels),
```

```
    D3D11_SDK_VERSION,      // This should always be D3D11_SDK_VERSION.
    &device,                    // created device
    &m_featureLevel,            // The feature level of the device.
    &context                    // The corresponding immediate context.
);
```

- Use the APIs in Windows.ApplicationModel.Resources.Core to load the resources. Asset references should be generalized (no suffix), leaving out the feature level. However, unlike language and scale, the system does not automatically determine which feature level is optimal for a given display; that is left to you to determine based on code logic. Once you make that determination, use the APIs to inform the OS of the preferred feature level. The system will then be able to retrieve the correct asset based on that preference. Here is a code sample that shows how to inform your app of the current DirectX feature level for the platform:

```
C++

// Set the current UI thread's MRT ResourceContext's DXFeatureLevel
with the right DXFL.

Platform::String^ dxFeatureLevel;
    switch (m_featureLevel)
    {
    case D3D_FEATURE_LEVEL_9_1:
    case D3D_FEATURE_LEVEL_9_2:
    case D3D_FEATURE_LEVEL_9_3:
        dxFeatureLevel = L"DX9";
        break;
    case D3D_FEATURE_LEVEL_10_0:
    case D3D_FEATURE_LEVEL_10_1:
        dxFeatureLevel = L"DX10";
        break;
    default:
        dxFeatureLevel = L"DX11";
    }

    ResourceContext::SetGlobalQualifierValue(L"DXFeatureLevel",
dxFeatureLevel);
```

Note

> In your code, load the texture directly by name (or path below the feature level directory). Do not include either the feature level directory name or the suffix. For example, load "textures\coolsign.dds", not "dxfl-dx11\textures\coolsign.dds" or "textures\coolsign_dxfl-dx11.dds".

- Now use the ResourceManager to locate the file that matches current DirectX feature level. The **ResourceManager** returns a ResourceMap, which you query with ResourceMap::GetValue (or ResourceMap::TryGetValue) and a supplied ResourceContext. This returns a ResourceCandidate that most closely matches the DirectX feature level that was specified by calling SetGlobalQualifierValue.

```
C++
```

```
// An explicit ResourceContext is needed to match the DirectX feature
level for the display on which the current view is presented.

auto resourceContext = ResourceContext::GetForCurrentView();
auto mainResourceMap = ResourceManager::Current->MainResourceMap;

// For this code example, loader is a custom ref class used to load
resources.
// You can use the BasicLoader class from any of the 8.1 DirectX
samples similarly.


auto possibleResource = mainResourceMap->GetValue(
    L"Files/BumpPixelShader.cso",
    resourceContext
);
Platform::String^ resourceName = possibleResource->ValueAsString;
```

- In Visual Studio 2015, select **PROJECT->Store->Create App Package...** and create the package.

- Make sure that you enable app bundles in the package.appxmanifest manifest settings.

Related topics

- Defining app resources
- Packaging apps
- App packager (MakeAppx.exe)

UWP programming

Article • 10/20/2022

This section provides information about developing UWP games. Note that some of these articles are written in the context of creating a UWP game with DirectX.

Topic	Description
Audio for games	Describes the use of XAudio2 and Microsoft Media Foundation to add music and sound effects into a DirectX game.
Input for games	Learn about the different kinds of input devices for UWP games and how to implement them.
Missing .NET APIs in Unity and UWP	Learn about the missing .NET APIs when building UWP games in Unity, and workarounds for common issues.
Networking for games	Explains how to develop and incorporate networking features into a DirectX game.

Audio for games

Article • 10/20/2022

Learn how to develop and incorporate music and sounds into your DirectX game, and how to process the audio signals to create dynamic and positional sounds.

For audio programming, we recommend using either the XAudio2 library in DirectX, or the Windows Runtime Audio graphs APIs. We use XAudio2 here. XAudio2 is a low-level audio library that provides a signal processing and mixing foundation for games, and it supports a variety of formats.

You can also implement simple sounds and music playback with Microsoft Media Foundation. Microsoft Media Foundation is designed for the playback of media files and streams, both audio and video, but can also be used in games, and is particularly useful for cinematic scenes or non-interactive components of your game.

Concepts at a glance

Here are a few audio programming concepts we use in this section.

- Signals are the basic unit of sound programming, analogous to pixels in graphics. The digital signal processors (DSPs) that process them are like the pixel shaders of game audio. They can transform signals, or combine them, or filter them. By programming to the DSPs, you can alter your game's sound effects and music with as little or as much complexity as you need.
- Voices are the submixed composites of two or more signals. There are 3 types of XAudio2 voice objects: source, submix, and mastering voices. Source voices operate on audio data provided by the client. Source and submix voices send their output to one or more submix or mastering voices. Submix and mastering voices mix the audio from all voices feeding them, and operate on the result. Mastering voices write audio data to an audio device.
- Mixing is the process of combining several discrete voices, such as the sound effects and the background audio that are played back in a scene, into a single stream. Submixing is the process of combining several discrete signals, such as the component sounds of an engine noise, and creating a voice.
- Audio formats. Music and sound effects can be stored in a variety of digital formats for your game. There are uncompressed formats, like WAV, and compressed formats like MP3 and OGG. The more a sample is compressed -- typically designated by its bit rate, where the lower the bit rate is, the more lossy the compression -- the worse fidelity it has. Fidelity can vary across compression

schemes and bit rates, so experiment with them to find what works best for your game.

- Sample rate and quality. Sounds can be sampled at different rates, and sounds sampled at a lower rate have much poorer fidelity. The sample rate for CD quality is 44.1 Khz (44100 Hz). If you don't need high fidelity for a sound, you can choose a lower sample rate. Higher rates may be appropriate for professional audio applications, but you probably don't need them unless your game demands professional fidelity sound.
- Sound emitters (or sources). In XAudio2, sound emitters are locations that emit a sound, be it a mere blip of a background noise or a snarling rock track played by an in-game jukebox. You specify emitters by world coordinates.
- Sound listeners. A sound listener is often the player, or perhaps an AI entity in a more advanced game, that processes the sounds received from a listener. You can submix that sound into the audio stream for playback to the player, or you can use it to take a specific in-game action, like awakening an AI guard marked as a listener.

Design considerations

Audio is a tremendously important part of game design and development. Many gamers can recall a mediocre game elevated to legendary status just because of a memorable soundtrack, or great voice work and sound mixing, or overall stellar audio production. Music and sound define a game's personality, and establish the main motive that defines the game and makes it stand apart from other similar games. The effort you spend designing and developing your game's audio profile will be well worth it.

Positional 3D audio can add a level of immersion beyond that provided by 3D graphics. If you are developing a complex game that simulates a world, or which demands a cinematic style, consider using 3D positional audio techniques to really draw the player in.

DirectX audio development roadmap

XAudio2 conceptual resources

XAudio2 is the audio mixing library for DirectX, and is primarily intended for developing high performance audio engines for games. For game developers who want to add sound effects and background music to their modern games, XAudio2 offers an audio graph and mixing engine with low-latency and support for dynamic buffers, synchronous sample-accurate playback, and implicit source rate conversion.

Topic	Description
Introduction to XAudio2	The topic provides a list of the audio programming features supported by XAudio2.
Getting Started with XAudio2	This topic provides information on key XAudio2 concepts, XAudio2 versions, and the RIFF audio format.
Common Audio Programming Concepts	This topic provides an overview of common audio concepts with which an audio developer should be familiar.
XAudio2 Voices	This topic contains an overview of XAudio2 voices, which are used to submix, operate on, and master audio data.
XAudio2 Callbacks	This topic covers the XAudio 2 callbacks, which are used to prevent breaks in the audio playback.
XAudio2 Audio Graphs	This topic covers the XAudio2 audio processing graphs, which take a set of audio streams from the client as input, process them, and deliver the final result to an audio device.
XAudio2 Audio Effects	The topic covers XAudio2 audio effects, which take incoming audio data and perform some operation on the data (such as a reverb effect) before passing it on.
Streaming Audio Data with XAudio2	This topic covers audio streaming with XAudio2.
X3DAudio	this topic covers X3DAudio, an API used in conjunction with XAudio2 to create the illusion of a sound coming from a point in 3D space.
XAudio2 Programming Reference	This section contains the complete reference for the XAudio2 APIs.

XAudio2 "how to" resources

Topic	Description
How to: Initialize XAudio2	Learn how to initialize XAudio2 for audio playback by creating an instance of the XAudio2 engine, and creating a mastering voice.

Topic	Description
How to: Load Audio Data Files in XAudio2	Learn how to populate the structures required to play audio data in XAudio2.
How to: Play a Sound with XAudio2	Learn how to play previously-loaded audio data in XAudio2.
How to: Use Submix Voices	Learn how to set groups of voices to send their output to the same submix voice.
How to: Use Source Voice Callbacks	Learn how to use XAudio2 source voice callbacks.
How to: Use Engine Callbacks	Learn how to use XAudio2 engine callbacks.
How to: Build a Basic Audio Processing Graph	Learn how to create an audio processing graph, constructed from a single mastering voice and a single source voice.
How to: Dynamically Add or Remove Voices From an Audio Graph	Learn how to add or remove submix voices from a graph that has been created following the steps in How to: Build a Basic Audio Processing Graph.
How to: Create an Effect Chain	Learn how to apply an effect chain to a voice to allow custom processing of the audio data for that voice.
How to: Create an XAPO	Learn how to implement **IXAPO** to create an XAudio2 audio processing object (XAPO).
How to: Add Run-time Parameter Support to an XAPO	Learn how to add run-time parameter support to an XAPO by implementing the **IXAPOParameters** interface.
How to: Use an XAPO in XAudio2	Learn how to use an effect implemented as an XAPO in an XAudio2 effect chain.
How to: Use XAPOFX in XAudio2	Learn how to use one of the effects included in XAPOFX in an XAudio2 effect chain.
How to: Stream a Sound from Disk	Learn how to stream audio data in XAudio2 by creating a separate thread to read an audio buffer, and to use callbacks to control that thread.
How to: Integrate X3DAudio with XAudio2	Learn how to use X3DAudio to provide the volume and pitch values for XAudio2 voices as well as the parameters for the XAudio2 built-in reverb effect.

Topic	Description
How to: Group Audio Methods as an Operation Set	Learn how to use XAudio2 operation sets to make a group of method calls take effect at the same time.
Debugging Audio Glitches in XAudio2	Learn how to set the debug logging level for XAudio2.

Media Foundation resources

Media Foundation (MF) is a media platform for streaming audio and video playback. You can use the Media Foundation APIs to stream audio and video encoded and compressed with a variety of algorithms. It is not designed for real-time gameplay scenarios; instead, it provides powerful tools and broad codec support for more linear capture and presentation of audio and video components.

Topic	Description
About Media Foundation	This section contains general information about the Media Foundation APIs, and the tools available to support them.
Media Foundation: Essential Concepts	This topic introduces some concepts that you will need to understand before writing a Media Foundation application.
Media Foundation Architecture	This section describes the general design of Microsoft Media Foundation, as well as the media primitives and processing pipeline it uses.
Audio/Video Capture	This topic describes how to use Microsoft Media Foundation to perform audio and video capture.
Audio/Video Playback	This topic describes how to implement audio/video playback in your app.
Supported Media Formats in Media Foundation	This topic lists the media formats that Microsoft Media Foundation supports natively. (Third parties can support additional formats by writing custom plug-ins.)
Encoding and File Authoring	This topic describes how to use Microsoft Media Foundation to perform audio and video encoding, and author media files.

Topic	Description
Windows Media Codecs	This topic describes how to use the features of the Windows Media Audio and Video codecs to produce and consume compressed data streams.
Media Foundation Programming Reference	This section contains reference information for the Media Foundation APIs.
Media Foundation SDK Samples	This section lists sample apps that demonstrate how to use Media Foundation.

Windows Runtime XAML media types

If you are using DirectX-XAML interop, you can incorporate the Windows Runtime XAML media APIs into your UWP apps using DirectX with C++ for simpler game scenarios.

Topic	Description
Windows.UI.Xaml.Controls.MediaElement	XAML element that represents an object that contains audio, video, or both.
Audio, video, and camera	Learn how to incorporate basic audio and video in your Universal Windows Platform (UWP) app.
MediaElement	Learn how to play a locally-stored media file in your UWP app.
MediaElement	Learn how to stream a media file with low-latency in your UWP app.
Media casting	Learn how to use the Play To contract to stream media from your UWP app to another device.

Reference

- XAudio2 Introduction
- XAudio2 Programming Guide
- Microsoft Media Foundation overview

Related topics

- XAudio2 Programming Guide

Game broadcast and capture

Article • 10/20/2022

This section provides information for adding game capture and broadcast capabilities to a UWP app.

Game broadcasting

Currently, users must initiate game broadcasting using the system UI that is built into Windows, but starting with Windows 10, version 1709, apps can launch the system broadcasting UI and can receive notifications when broadcasting starts and stops. For more information, see Manage game broadcasting.

Game capture

Starting with Windows 10, version 1709, UWP apps can record audio and video captures of gameplay, capture screenshots, and submit gameplay metadata to be embedded in captured and broadcast video streams, allowing your app and others to create dynamic experiences that are synchronized to gameplay events. For more information, see Capture game audio, video, screenshots, and metadata.

See Also

- Gaming

Manage game broadcasting

Article • 10/20/2022

This article shows you how to manage game broadcasting for a UWP app. Users must initiate broadcasting by using the system UI that is built into Windows, but starting with Windows 10, version 1709, apps can launch the system broadcasting UI and can receive notifications when broadcasting starts and stops.

Add the Windows Desktop Extensions for the UWP to your app

The APIs for managing app broadcasting, found in the Windows.Media.AppBroadcasting namespace, are not included in the Universal API contract. To access the APIs, you must add a reference to the Windows Desktop Extensions for the UWP to your app with the following steps.

1. In Visual Studio, in **Solution Explorer**, expand your UWP project and right-click **References** and then select **Add Reference...**.
2. Expand the **Universal Windows** node and select **Extensions**.
3. In the list of extensions check the checkbox next to the **Windows Desktop Extensions for the UWP** entry that matches the target build for your project. For the app broadcast features, the version must be 1709 or greater.
4. Click **OK**.

Launch the system UI to allow the user to initiate broadcasting

There are several reasons that your app may not currently be able to broadcast, including if the current device doesn't meet the hardware requirements for broadcasting or if another app is currently broadcasting. Before launching the system UI, you can check to see if your app is currently able to broadcast. First, check to see if the broadcast APIs are available on the current device. The APIs are not available on devices running an OS version earlier than Windows 10, version 1709. Rather than check for a specific OS version, use the ApiInformation.IsApiContractPresent method to query for the *Windows.Media.AppBroadcasting.AppBroadcastingContract* version 1.0. If this contract is present, then the broadcasting APIs are available on the device.

Next, get an instance of the AppBroadcastingUI class by calling the factory method GetDefault on PC, where there is a single user signed in at a time. On Xbox, where

multiple users can be signed in, call **GetForUser** instead. Then call **GetStatus** to get the broadcasting status of your app.

The **CanStartBroadcast** property of the **AppBroadcastingStatus** class tells you whether the app can currently start broadcasting. If not, you can check the **Details** property to determine the reason broadcasting is not available. Depending on the reason, you may want to display the status to the user or show instructions for enabling broadcasting.

C++

```
// Verify that the edition supports the AppBroadcasting APIs
if (!Windows::Foundation::Metadata::ApiInformation::IsApiContractPresent(
    "Windows.Media.AppBroadcasting.AppBroadcastingContract", 1, 0))
{
    return;
}

// On PC, call GetDefault because there is a single user signed in.
// On Xbox, call GetForUser instead because multiple users can be signed in.
AppBroadcastingUI^ broadcastingUI = AppBroadcastingUI::GetDefault();
AppBroadcastingStatus^ broadcastingStatus = broadcastingUI->GetStatus();

if (!broadcastingStatus->CanStartBroadcast)
{
    AppBroadcastingStatusDetails^ details = broadcastingStatus->Details;

    if (details->IsAnyAppBroadcasting)
    {
        UpdateStatusText("Another app is currently broadcasting.");
        return;
    }

    if (details->IsCaptureResourceUnavailable)
    {
        UpdateStatusText("The capture resource is currently unavailable.");
        return;
    }

    if (details->IsGameStreamInProgress)
    {
        UpdateStatusText("A game stream is currently in progress.");
        return;
    }

    if (details->IsGpuConstrained)
    {
        // Typically, this means that the GPU software does not include an
H264 encoder
        UpdateStatusText("The GPU does not support broadcasting.");
        return;
    }
```

```
    // Broadcasting can only be started when the application's window is the
active window.
    if (details->IsAppInactive)
    {
        UpdateStatusText("The app window to be broadcast is not active.");
        return;
    }

    if (details->IsBlockedForApp)
    {
        UpdateStatusText("Broadcasting is blocked for this app.");
        return;
    }

    if (details->IsDisabledByUser)
    {
        UpdateStatusText("The user has disabled GameBar in Windows
Settings.");
        return;
    }

    if (details->IsDisabledBySystem)
    {
        UpdateStatusText("Broadcasting is disabled by the system.");
        return;
    }
    return;
}
```

Request that the app broadcast UI be shown by the system by calling ShowBroadcastUI.

> ⓘ Note
>
> The **ShowBroadcastUI** method represents a request that may not succeed, depending on the current state of the system. Your app should not assume that broadcasting has begun after calling this method. Use the IsCurrentAppBroadcastingChanged event to be notified when broadcasting starts or stops.

C++

```
broadcastingUI->ShowBroadcastUI();
```

Receive notifications when broadcasting starts and stops

Register to receive notifications when the user uses the system UI to start or stop broadcasting your app by initializing an instance of **AppBroadcastingMonitor** class and registering a handler for the **IsCurrentAppBroadcastingChanged** event. As discussed in the previous section, be sure to use the **ApiInformation.IsApiContractPresent** at some point to verify that the broadcasting APIs are present on the device before attempting to use them.

C++

```
if (Windows::Foundation::Metadata::ApiInformation::IsApiContractPresent(
    "Windows.Media.AppBroadcasting.AppBroadcastingContract", 1, 0))
{
    m_appBroadcastMonitor = ref new AppBroadcastingMonitor();

    m_appBroadcastMonitor->IsCurrentAppBroadcastingChanged +=
        ref new TypedEventHandler<AppBroadcastingMonitor^,
Platform::Object^>(this, &App::OnIsCurrentAppBroadcastingChanged);
}
```

In the handler for the **IsCurrentAppBroadcastingChanged** event, you may want to update your app's UI to reflect the current broadcasting state.

C++

```
void App::OnIsCurrentAppBroadcastingChanged(AppBroadcastingMonitor^ sender,
Platform::Object^ args)
{
    if (sender->IsCurrentAppBroadcasting)
    {
        UpdateStatusText("App started broadcasting.");
    }
    else
    {
        UpdateStatusText("App stopped broadcasting.");
    }
}
```

Related topics

- Gaming

Capture game audio, video, screenshots, and metadata

Article • 10/20/2022

This article describes how to capture game video, audio, and screenshots, and how to submit metadata that the system will embed in captured and broadcast media, allowing your app and others to create dynamic experiences that are synchronized to gameplay events.

There are two different ways that gameplay can be captured in a UWP app. The user can initiate capture using the built-in system UI. Media that is captured using this technique is ingested into the Microsoft gaming ecosystem, can be viewed and shared through first-party experiences such as the Xbox app, and is not directly available to your app or to users. The first sections of this article will show you how to enable and disable system-implemented app capture and how to receive notifications when app capture starts or stops.

The other way to capture media is to use the APIs of the **Windows.Media.AppRecording** namespace. If capturing is enabled on the device, your app can start capturing gameplay and then, after some time has passed, you can stop the capture, at which point the media is written to a file. If the user has enabled historical capture, then you can also record gameplay that has already occurred by specifying a start time in the past and a duration to record. Both of these techniques produce a video file that can be accessed by your app, and depending on where you choose to save the files, by the user. The middle sections of this article walk you through the implementation of these scenarios.

The **Windows.Media.Capture** namespace provides APIs for creating metadata that describes the gameplay being captured or broadcast. This can include text or numeric values, with a text label identifying each data item. Metadata can represent an "event" which occurs at a single moment, such as when the user finishes a lap in a racing game, or it can represent a "state" that persists over a span of time, such as the current game map the user is playing in. The metadata is written to a cache that is allocated and managed for your app by the system. The metadata is embedded into broadcast streams and captured video files, including both the built-in system capture or custom app capture techniques. The final sections of this article show you how to write gameplay metadata.

> **Note**

Because the gameplay metadata can be embedded in media files that can potentially be shared over the network, out of the user's control, you should not include personally identifiable information or other potentially sensitive data in the metadata.

Enable and disable system app capture

System app capture is initiated by the user with the built-in system UI. The files are ingested by the Windows gaming ecosystem and is not available to your app or the user, except for through first party experiences like the Xbox app. Your app can disable and enable system-initiated app capture, allowing you to prevent the user from capturing certain content or gameplay.

To enable or disable system app capture, simply call the static method AppCapture.SetAllowedAsync and passing **false** to disable capture or **true** to enable capture.

C++

```
Windows::Media::Capture::AppCapture::SetAllowedAsync(allowed);
```

Receive notifications when system app capture starts and stops

To receive a notification when system app capture begins or ends, first get an instance of the AppCapture class by calling the factory method GetForCurrentView. Next, register a handler for the CapturingChanged event.

C++

```
Windows::Media::Capture::AppCapture^ appCapture =
Windows::Media::Capture::AppCapture::GetForCurrentView();
appCapture->CapturingChanged +=
    ref new TypedEventHandler<Windows::Media::Capture::AppCapture^,
Platform::Object^>(this, &App::OnCapturingChanged);
```

In the handler for the **CapturingChanged** event, you can check the IsCapturingAudio and the IsCapturingVideo properties to determine if audio or video are being captured respectively. You may want to update your app's UI to indicate the current capturing status.